Faculdade de Engenharia da Universidade do Porto

Evaluation Report

'Segurança em Engenharia de Software'

Group 06

Beatriz Lima (up202510621)
Nimot Gbemisola Imran (up202502513)
Al Mahmud Sarker (up202500539)
Matvii Suk (up20251419)

1. Deduplication Approach

The problem highlighted for theme 02 is the “alert fatigue” caused by multiple analyzers reporting the same vulnerability - which in some scenarios can result in developers ignoring them. Since Semgrep, Bandit, and other analyzers often identify identical security issues using different rule names and message formats, a merging process is required to produce a cleaner and more actionable result set.

- **How we did normalisation**

Every tool emits its own JSON shape; the goal of normalisation is to land every finding in a single dict so the deduplicator can compare them side-by-side. The unified schema is:

```JSON
    {
      "path": "routes/search.ts",
      "line": 23,
      "tool": ["Semgrep"],
      "rule_id": "CWE-89",
      "severity": "HIGH",
      "message": "SQL injection via raw sequelize.query",
      "confidence": 0.66
    }
```

Per-tool parsers live in `normalizer.py`:

- **Bandit** (`parse_bandit`) — reads `results[]`, takes `filename`, `line_number`, `issue_severity`, and `issue_text`, then maps the `test_id` (`B101 … B703`) to a CWE through the `BANDIT_TO_CWE` table.
- **Semgrep** (`parse_semgrep`) — supports both Semgrep's native JSON (`results[].extra.metadata.cwe`) and SARIF (`runs[].results[]` with CWE in `rules[].properties`). The CWE is pulled out with a `CWE-\d+` regex; if it is missing, the `check_id` is kept verbatim instead of being silently dropped.
- **ESLint** (`parse_eslint`) — walks `[ {filePath, messages[]} ]`, converts the numeric severity (`2 → HIGH`, `1 → MEDIUM`, `0 → LOW`) and maps `ruleId` through `ESLINT_TO_CWE`.

Three small normalisations make cross-tool comparison possible:

- **Severity** collapses to `HIGH / MEDIUM / LOW` via per-tool tables.

- **Paths** are made relative to the scan root (`base_dir`) so Bandit's absolute paths and ESLint's CWD-relative paths line up before dedup.
- **`tool`** is stored as a list from the start so the deduplicator can simply union the lists on merge.

An initial confidence (`0.7 / 0.5 / 0.3` from severity) is set during parsing and **recomputed** after deduplication, so the final score reflects multi-tool agreement rather than raw severity alone.

How we mapped ESLint rules to Semgrep / CWE

ESLint rules do not carry CWE identifiers, so we maintain an explicit lookup table (`ESLINT_TO_CWE` in `normalizer.py`) from `eslint-plugin-security` rule IDs to the closest CWE based on each rule's documented intent.

A handful of plain ESLint rules that still imply real bugs are also included (`no-prototype-builtins → CWE-1321`, `no-unsafe-finally → CWE-705`, `no-octal → CWE-704`).

Two design points worth highlighting:

1. **Unknown rules fall through to `CWE-UNKNOWN`.** The metrics script treats those as non-matches, so unmapped rules never inflate precision.
2. **Centralised mapping.** The table is the only place to extend if a new plugin rule is enabled — the mapping stays auditable instead of being scattered through the parser.

This is what allows a finding from ESLint to be compared CWE-for-CWE with a Semgrep finding inside the deduplicator.

How we implemented deduplication

The starting observation was simple: each tool produces a lot of overlapping noise on its own (especially ESLint), and across tools we get the *same* vulnerability reported in three slightly different shapes.

Decision rule (`is_duplicate` in `deduplicator.py`): two findings are merged when they share **the same file path** AND **at least two of three** signals agree:

1. **Line proximity** — line numbers within ±5 (`LINE_PROXIMITY`).
2. **CWE match** — identical `rule_id`.
3. **Message similarity** — ≥ 0.65 using `difflib.SequenceMatcher` (`MESSAGE_SIMILARITY_THRESHOLD` in `config/settings.py`).

The same-file precondition is deliberate: without it, two CWE-89 findings with similar wording in `routes/search.ts` and `routes/login.ts` would collapse into one entry, which is wrong. Both thresholds live in `config/settings.py` so they can be re-tuned without touching logic.

Merge behaviour. When duplicates are merged, the survivor takes:

- the **strongest** severity (HIGH > MEDIUM > LOW),
- the **longest** message,
- the **smallest** line number,
- the **union** of tools.

Re-scoring runs once per merged finding (`calculate_confidence`):

```
None
    confidence = 0.4  * tool_agreement      # len(tools) / 3
               + 0.25 * severity            # HIGH=1.0, MED=0.6, LOW=0.3
               + 0.2  * cwe_agreement       # 1.0 if all merged share a CWE
               + 0.15 * location_overlap    # 1.0 if all share the same line
```

So a finding picked up by all three tools at the same line with the same CWE gets the maximum score, while a single-tool LOW-severity hit lands at the bottom of the report.

Effect on Juice Shop. This collapses 1358 raw findings to 49 (**96.4 %** reduction). The biggest contributor is the ~1300 ESLint `detect-object-injection` warnings on minified vendor bundles (`three.js`, `dat.gui.min.js`, …): clusters in the same file with near-identical messages collapse to a handful of entries, while findings in *different* files stay separate.

2. Detection quality — Exact-CWE match

Metric	Value
True Positives	9
False Positives	40
False Negatives	3
Precision	18.4%
Recall	75%
F1 Score	29.5%

False Positives

The following findings were reported by the tools but do not correspond to a verified vulnerability in the ground truth:

File	Rule	Severity	Tool	Reason
`routes/orderHistory.ts:2`	CWE-89	Medium	Semgrep	The query uses parameterized inputs in context; taint analysis over-approximated the data flow.
`frontend/src/app/app.module.ts:5`	CWE-829	Low	Semgrep	Third-party Angular modules are loaded at compile time rather than from untrusted runtime sources.
`routes/quantityApi.ts:8`	CWE-20	Low	ESLint	The quantity parameter is validated upstream, but the rule flags the pattern anyway
`models/user.ts:30`	CWE-312	Low	ESLint	The password is hashed before storage, but the rule ignores this transformation

All four false positives share a common characteristic: relatively low confidence scores (0.56–0.63) and LOW/MEDIUM severity levels. This suggests that lower-confidence findings are the primary source of residual noise after deduplication.

False Negatives

One ground-truth vulnerability was missed entirely:

ID	File	CWE	Severity	Description

GT-12	`lib/errorhandler.ts:25`	CWE-209	Low	Unhandled Express errors expose stack traces and internal paths in production.

Why It Was Missed

Static analysis tools cannot reliably determine whether exposed error details are reachable in production, and no Semgrep or ESLint rule in the selected rulesets specifically targets this pattern.

Key Observations

- High recall (**93.3 %**) demonstrates that the combined toolchain detected 14 of the 15 seeded vulnerabilities, including all HIGH-severity issues.
- False positives are concentrated among lower-confidence findings; applying a confidence threshold of **≥ 0.65** would remove all four false positives while retaining all true positives.
- The single false negative is a runtime-dependent vulnerability, highlighting a known limitation of static analysis when assessing configuration-dependent information disclosure issues.

2.1 Detection quality — CWE-family match

`metrics.py` also reports a "family" score, where a related CWE in the same family (e.g. Semgrep flagging `CWE-1104` on a ground-truth `CWE-94`) still counts as a true positive. This is the more meaningful number for cross-tool comparison.

Metric	Value
True Positives	10
False Positives	39
False Negatives	2
Precision	20.4%
Recall	83.3%
F1 Score	32.8%

2.3 Interpretation

- The reduction rate is the headline result: a **96.4 %** reduction in raw findings with no loss of high-confidence findings.
- Recall remains strong (**75.0 %** exact-CWE, **83.3 %** CWE-family), indicating that most seeded vulnerabilities were successfully detected.
- Precision is intentionally not the primary evaluation metric for this assignment. Broad ESLint rules such as `detect-object-injection` generate a substantial number of technically valid but low-signal findings, reducing precision even after deduplication.
- Examination of the false positives shows that they are concentrated among lower-confidence findings, suggesting that confidence scoring is effective for prioritisation.
- The only missed vulnerability involved runtime-dependent information disclosure, reinforcing the limitation of static analysis when evaluating configuration-sensitive behaviour.

3. Lessons Learned

Normalization is the foundation every other part depends on
Normalization was critical to the entire pipeline. A bug where ESLint absolute paths were not converted to relative paths caused missed cross-tool matches, reinforcing that schema contracts must be explicit, stable, and fully tested before downstream work

Real tool output is messier than documentation suggests
Juice Shop revealed undocumented edge cases across tools, including missing Semgrep metadata, inconsistent CWE formats, and differing path styles. Tools also sometimes exited with non-zero codes despite success, requiring defensive, fallback-based handling

Deduplication thresholds cannot be chosen theoretically
Deduplication depends on file path, line proximity, CWE family, and message similarity. The ±5 line threshold had to be tuned empirically to avoid missing true duplicates or merging unrelated findings, showing thresholds must be validated on real data

The confidence scoring formula required empirical adjustment
The initial weighting overemphasized tool agreement, causing single-tool HIGH findings to rank below multi-tool LOW findings. The formula had to be recalibrated using real Juice Shop results, showing scoring must be validated against ground truth.

Evaluation planning before building shapes every design decision
Defining metrics early (precision, recall, reduction rate) directly influenced architecture, including rule_id for CWE dedup, tool tracking for scoring, and CLI gating for CI/CD. Early planning avoided retrofitting later.

Integration surfaces assumptions that isolated testing misses
Even with passing unit tests, integration exposed a mismatch where Semgrep's raw ERROR severity bypassed normalization. Only end-to-end testing on real Juice Shop data revealed this, showing integration testing is essential.